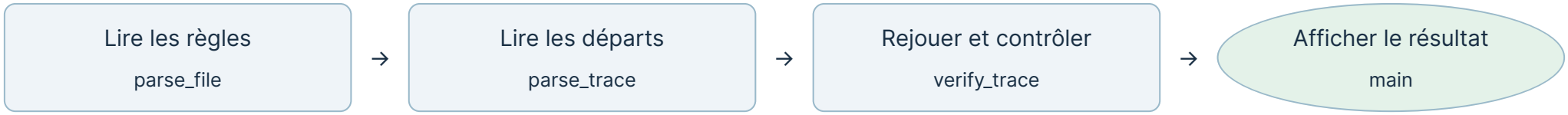
Comprendre krpsim_verif

Arbre de décision expliqué pas à pas

Lire les décisions en français.
Retrouver les fonctions et les variables dans le code.



> **La question centrale**
> « Chaque opération annoncée peut-elle réellement démarrer avec les ressources disponibles à ce moment-là ? »

Pour un développeur débutant qui souhaite comprendre son programme construit avec l'aide de l'IA.

Édition du 7 novembre 2026 · krpsim 0.1.1 · révision 268075e
Sources locales examinées ; programme conservé. Les modifications locales du Makefile et des documents existants sont hors de ce guide.

Votre parcours de lecture

Les entrées ci-dessous sont cliquables. Les identifiants D01 à D60 relient les décisions.

Un atelier, des recettes et un carnet 3

Entrer dans le programme 6

Choisir le parcours : trace vide ou départs 10

Quelles ressources manquent exactement ? 13

Consommer, produire et avancer dans la liste 15

L'atelier utilisé dans les exemples 18

Comparer les traces : quand faut-il refuser ? 20

Configuration : choisir le fichier 26

Annexe : préparer l'objet retourné 43

Retrouver les preuves et régénérer le guide 49

À vous de suivre les décisions 51

Le parcours complet en une vue 5

Lire chaque ligne de trace 8

Ajouter les résultats, puis trouver l'opération 12

Ce que le programme mémorise 14

Construire et afficher le résultat 16

Suivre les stocks, départ par départ 19

Les limites réelles du programme 24

Les boucles : où revient-on ? 42

Corrections apportées au document précédent 48

Ce qui a été vérifié 50

Un atelier, des recettes et un carnet

Le vocabulaire français reste stable dans tous les schémas.

Un cycle est une unité de la simulation : ce n'est pas nécessairement une seconde. Une trace ne liste pas les fins ; le programme les calcule avec le départ et la durée.

Mot du guide	Ce que cela veut dire	Nom exact dans le code
Ressource / stock	Du bois, des clous ; leur quantité disponible.	resource / stocks
Opération / besoins	Une recette ; ce qu'elle consomme au démarrage.	Process / process.needs
Résultats / durée	Ce qu'elle produit à sa fin ; combien de cycles elle prend.	process.results / process.delay
Cycle / départ	Un repère de temps ; le lancement annoncé d'une opération.	entry.cycle / TraceEntry
Configuration	Le fichier des stocks initiaux, opérations et objectifs.	Config / config
Trace	Le carnet ordonné des départs annoncés : cycle:nom.	list[TraceEntry] / trace
Opérations en cours	Les opérations lancées dont les résultats sont encore attendus.	state.running

Lire le schéma et le code côte à côte

La question française explique le sens ; la ligne Python permet de retrouver le contrôle.

Conventions visuelles

Ovale : début ou fin du parcours.

Rectangle : action à effectuer.

Losange : question à trancher.

Oui / Non : suivre la sortie portant cette réponse. La couleur seule n'indique jamais la réponse.

Dxx : renvoi cliquable vers une autre décision. Les sous-schémas sont reliés par ces renvois, sans forcer un arbre géant.

Repères du code

entry.cycle : moment du départ lu.

state.previous_cycle : départ précédent ; vaut 0 au début.

state.last_cycle : plus grand départ ou fin déjà pris en compte.

state.sequence : compteur qui départage les fins au même moment.

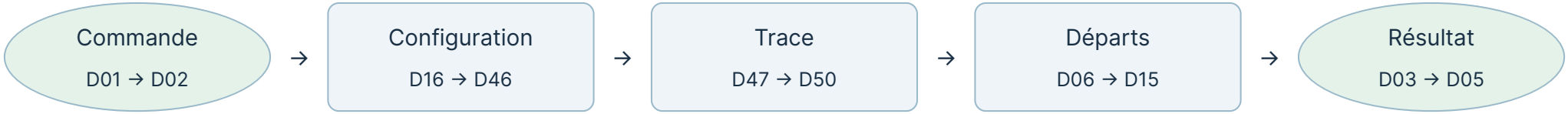
missing : ressources manquantes et quantités à ajouter pour pouvoir démarrer.

sim : objet contenant l'état final renvoyé.

> **Lecture d'une fiche** : question → réponse → action ou renvoi. La ligne « Pourquoi » explique la règle ; l'exemple aide à l'appliquer. Les références sont relatives à la racine du dépôt.

Le parcours complet en une vue

main → _verify_inputs → verify_files → parse_file → parse_trace → verify_trace



Avant de rejouer

1. La commande lit ses arguments.

2. Le fichier de configuration est entièrement validé.

3. Le fichier de trace est entièrement lu et son format vérifié.

4. Les départs valides sur la forme sont rejoués dans l'ordre.

Après chaque départ

Vérifier le temps → ajouter les productions terminées → trouver l'opération → vérifier ses ressources → consommer → produire ou programmer la fin.

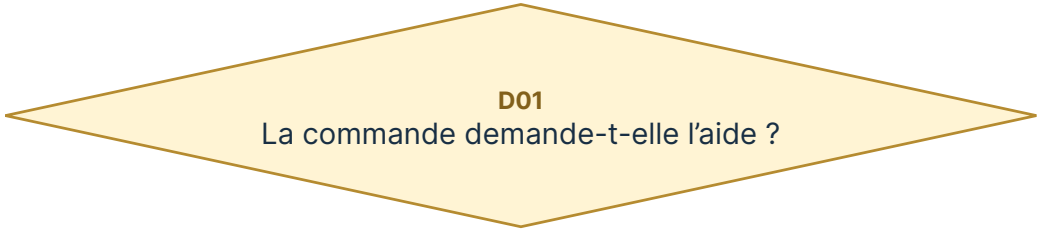
Après le dernier départ, finaliser les productions jusqu'au seuil du code, construire **Simulator**, puis afficher les stocks.

> **L'ordre change l'erreur visible.** Une ligne mal écrite en fin de fichier est détectée avant un manque de ressources au premier départ : le programme finit la lecture de la trace avant de la rejouer.

Sources : src/krpsim_verif/cli.py:47-101 ; verifier.py:178-280. Les flèches décrivent les appels ; les exceptions reviennent au diagnostic D03 / D04.

Entrer dans le programme

main / build_parser · Avant toute lecture de fichier.



Oui → Afficher l'aide ; arrêt normal, code 0.

Non → Contrôler les arguments : D02.

main → argparse · src/krpsim_verif/cli.py:94
parse_args(argv) : option -h / --help

Pourquoi : L'aide explique la commande sans ouvrir de fichier.
Exemple : krpsim_verif --help affiche les deux chemins attendus.



Oui → Afficher usage et erreur ; arrêt, code 2.

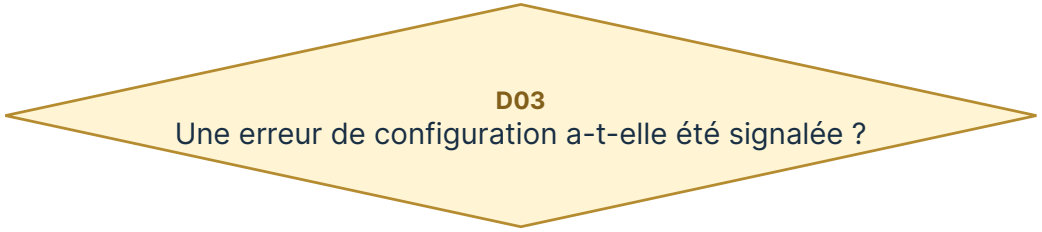
Non → Appeler _verify_inputs → verify_files.

build_parser / main · src/krpsim_verif/cli.py:41-44,94
parse_args(argv) : erreur d'arguments

Pourquoi : Il faut indiquer un fichier de configuration et un fichier de trace.
Exemple : Un chemin manquant ou une option inconnue provoque une erreur.

Transformer les erreurs en messages

_verify_inputs · Ces questions représentent le choix des blocs except.

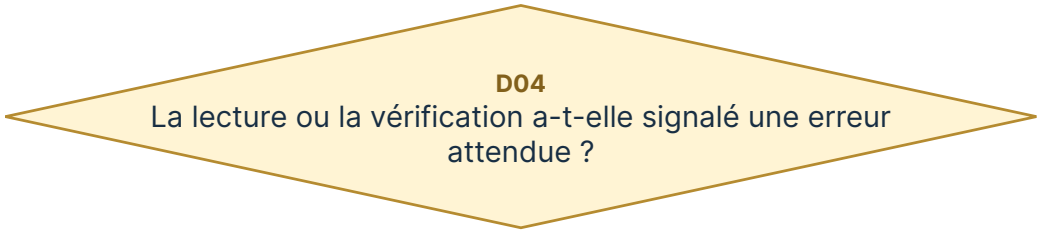


Oui → Afficher invalid config: ... ; renvoyer (None, 1) → D05.

Non → Tester l'autre catégorie d'erreurs : D04.

_verify_inputs · src/krpsim_verif/cli.py:50-55
except ParseError as exc

Pourquoi : La commande transforme une exception précise en message lisible.
Exemple : Un stock déclaré deux fois produit invalid config.



Oui → Afficher invalid trace: ... ; renvoyer (None, 1) → D05.

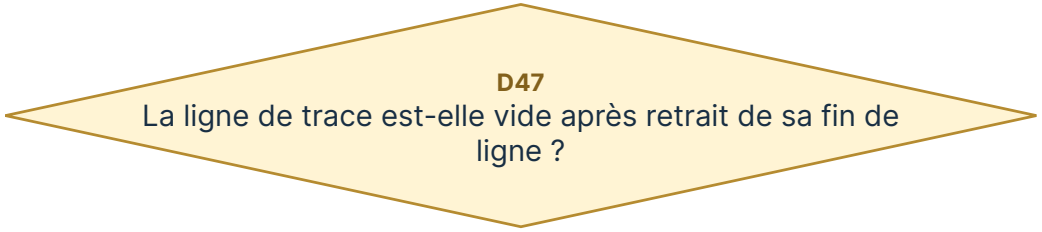
Non → Sans erreur : trace is valid → D05. Exception non prévue : propagation.

_verify_inputs · src/krpsim_verif/cli.py:56-60
except (OSError, TraceError) as exc

Pourquoi : Seules les exceptions prévues ici deviennent ce diagnostic.
Exemple : Une trace absente déclenche OSError. Un encodage de trace invalide échappe à ce traitement.

Lire chaque ligne de trace

parse_trace → _parse_trace_line · Ouvrir en UTF-8 ; idx compte les lignes physiques.

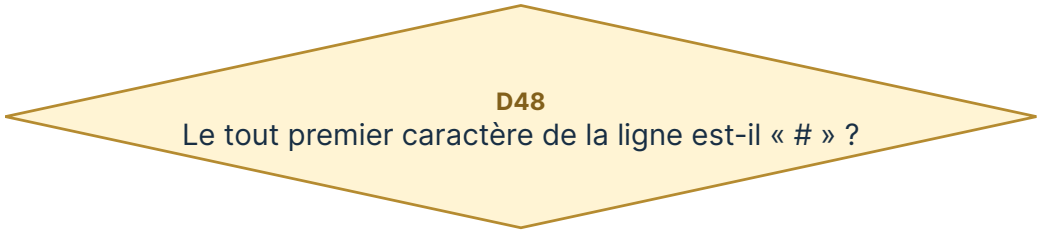


Oui → Refuser : empty trace line idx → D04.

Non → Tester le commentaire → D48.

_parse_trace_line · src/krpsim_verif/verifier.py:88,100-101
not line

Pourquoi : Une ligne vide est une erreur, contrairement à un fichier sans aucune ligne.
Exemple : Un fichier contenant seulement un saut de ligne est refusé ; un fichier de zéro octet va à D06.



Oui → Ignorer ; lire la ligne suivante. À la fin : D06.

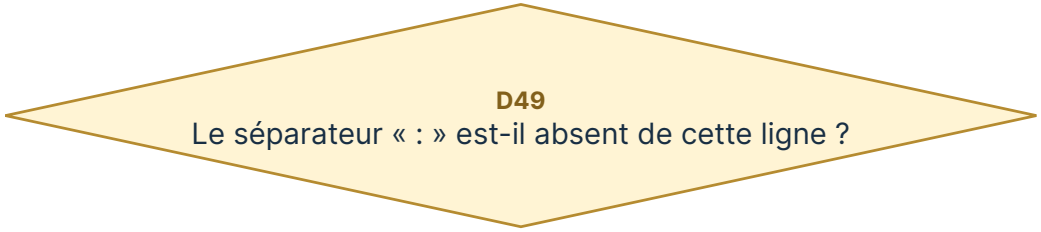
Non → Chercher le séparateur → D49.

_parse_trace_line · src/krpsim_verif/verifier.py:102-103
line.startswith("#")

Pourquoi : La trace ne supprime pas les espaces avant de chercher un commentaire.
Exemple : # atelier est ignoré ; « # atelier » n'est pas reconnu comme commentaire.

Séparer le moment et le nom

_parse_trace_line · Après une ligne acceptée ou ignorée, lire la suivante ; à la fin, D06.



Oui → Refuser : invalid trace line idx → D04.

Non → Couper au premier : ; vérifier cycle et nom → D50.

_parse_trace_line · src/krpsim_verif/verifier.py:104-106
":" not in line

Pourquoi : Le premier deux-points sépare le moment du départ et le nom de l'opération.
Exemple : 0scier est refusé ; 0:scier:extra garde scier:extra comme nom entier.



Oui → Refuser : invalid trace line idx → D04.

Non → Convertir le cycle → D51 ; ajouter TraceEntry ; ligne suivante.

_parse_trace_line · src/krpsim_verif/verifier.py:107-110
not cycle_str.isdigit() or not name

Pourquoi : Le format est contrôlé ici ; l'existence de l'opération est vérifiée plus tard en D11.
Exemple : x:scier et 0: sont refusés ; 00:scier devient le cycle 0.

Choisir le parcours : trace vide ou départs

verify_trace → _verify_empty_trace · Ne pas confondre fichier vide et ligne vide.

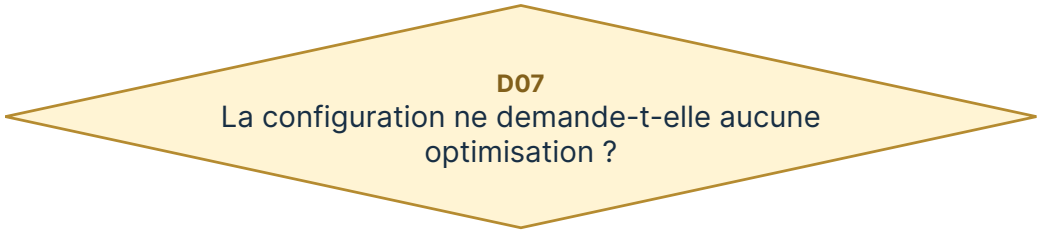


Oui → Appeler _verify_empty_trace → D07.

Non → Copier les stocks ; parcourir les départs → D09.

verify_trace · src/krpsim_verif/verifier.py:237-246
not trace

Pourquoi : Un fichier sans départ suit une règle particulière.
Exemple : Un fichier vide et un fichier contenant seulement # commentaire donnent tous deux une liste vide.



Oui → Tester les opérations avec les stocks initiaux : D08.

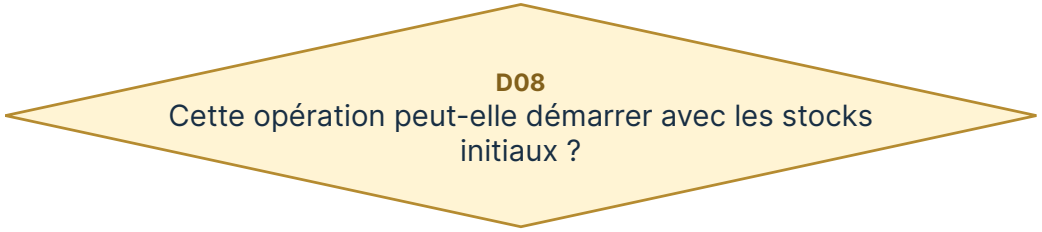
Non → Accepter la trace vide ; Simulator(config), temps 0.

_verify_empty_trace · src/krpsim_verif/verifier.py:115-120
not config.optimize

Pourquoi : La présence d'une directive optimize non vide change la règle de trace vide.
Exemple : optimize:(chaise) suffit à accepter une trace vide, même si une opération pourrait démarrer.

Du stock initial au premier contrôle

_verify_empty_trace pour D08 ; _verify_entry pour D09 : deux parcours distincts.



Oui → Refuser immédiatement : TraceError("empty trace").

Non → Tester l'opération suivante ; si aucune ne peut démarrer, accepter.

_verify_empty_trace · src/krpsim_verif/verifier.py:116-119
```
all(config.stocks.get(n, 0) >= q for n, q in proc.needs.items())
```

Pourquoi : Sans optimisation, une trace vide est refusée si du travail est déjà possible.
Exemple : Il faut 1 bois : stock 2 → refus ; stock 0 et aucune autre opération possible → acceptation.



Oui → Refuser : cycle … is before cycle … → D04.

Non → Ajouter les productions terminées : _complete_until → D10.

_verify_entry · src/krpsim_verif/verifier.py:186-191
```
entry.cycle < state.previous_cycle
```

Pourquoi : Les moments annoncés ne doivent pas reculer. Au premier départ, la référence vaut 0.
Exemple : Après le cycle 8 : 5 est refusé ; 8 est autorisé par ce contrôle.

Ajouter les résultats, puis trouver l'opération

_verify_entry appelle _complete_until puis _resolve_process, dans cet ordre.

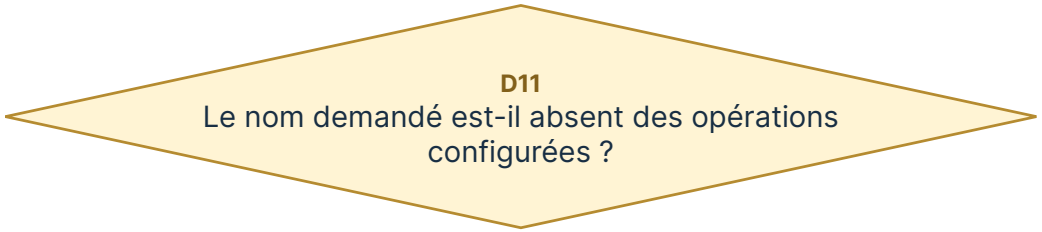


Oui → Retirer la fin la plus proche, ajouter ses résultats ; recommencer D10.

Non → Renvoyer last_completion : dernière fin ajoutée, ou 0.

_complete_until · src/krpsim_verif/verifier.py:130-136
running and running[0][0] <= cycle

Pourquoi : Les ressources produites deviennent disponibles avant de contrôler le nouveau départ.
Exemple : scier finit au cycle 2 : ses planches sont disponibles pour un départ au cycle 2.



Oui → Refuser : unknown process … in trace → D04.

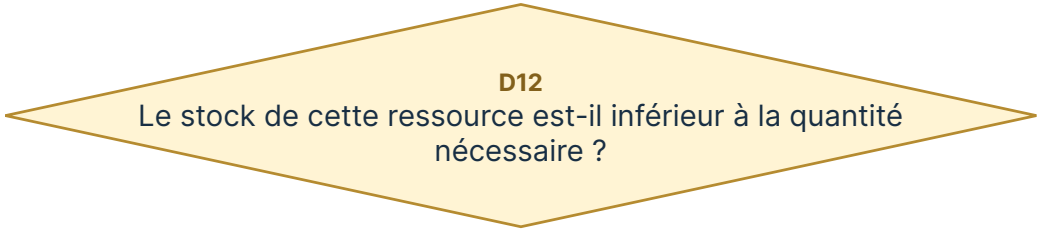
Non → Renvoyer process ; vérifier ses ressources → D12.

_resolve_process · src/krpsim_verif/verifier.py:142-145
process is None

Pourquoi : Chaque départ doit citer une opération qui existe dans la configuration.
Exemple : 0:inconnue est bien écrit, mais aucune opération de ce nom n'existe.

Quelles ressources manquent exactement ?

_missing_resources calcule les manques ; _verify_entry décide ensuite du refus.

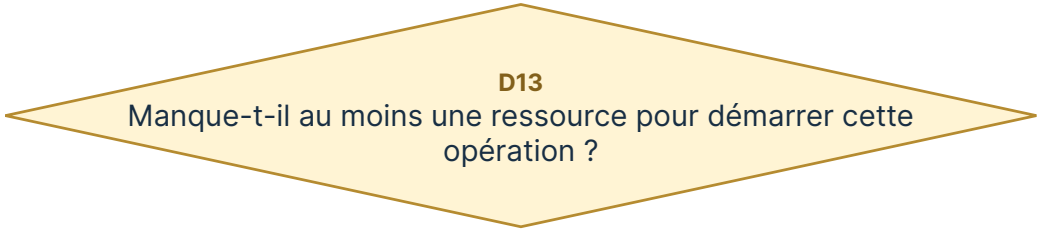


Oui → Ajouter le manque : quantité nécessaire - stock ; ressource suivante.

Non → Ne rien ajouter ; ressource suivante. Après toutes : D13.

_missing_resources · src/krpsim_verif/verifier.py:151-155
stocks.get(resource, 0) < quantity

Pourquoi : Chaque ressource est contrôlée séparément. Une ressource absente du stock vaut 0.
Exemple : 3 planches nécessaires, 2 disponibles : ajouter planche: 1 à missing.



Oui → Refuser : ligne, cycle, opération et quantités manquantes → D04.

Non → Appeler _consume_and_schedule : consommer les besoins → D14.

_verify_entry · src/krpsim_verif/verifier.py:194-200
if missing

Pourquoi : Avoir assez de clous ne compense pas un manque de planches.
Exemple : Une chaise demande 3 planches et 4 clous ; avec 2 planches et 10 clous, il manque 1 planche.

Ce que le programme mémorise

_VerificationState et _consume_and_schedule · Les valeurs changent au fil du rejeu.

Au début d'une trace non vide

```
state = _VerificationState(config.stocks.copy(), [])
```

stocks : copie des quantités initiales ; la configuration est conservée.

running : liste vide des fins attendues.

previous_cycle = last_cycle = sequence = 0.

Consommer au départ

```
for resource, quantity in process.needs.items():
    state.stocks[resource] -= quantity
```

On retire toutes les ressources nécessaires avant de produire quoi que ce soit, même pour une opération de durée nulle.

Produire immédiatement ou plus tard

Durée nulle : pour chaque résultat, ajouter sa quantité au stock courant (0 si absent).

```
state.stocks[resource] = \
    state.stocks.get(resource, 0) + quantity
```

Durée positive : incrémenter **sequence**, puis ajouter la fin à la file avec **heapq.heappush** :

```
(entry.cycle + process.delay,
 state.sequence, process)
```

La file donne d'abord la fin la plus proche. Si deux fins sont égales, **sequence** les départage sans comparer les objets Process. **heapq.heappop** retire la prochaine fin.

> Ajouter une production terminée ne retire aucun besoin : les besoins ont déjà été consommés au départ. Une opération sans résultat peut néanmoins faire avancer le dernier cycle.

Sources : src/krpsim_verif/verifier.py:51-59,123-136,158-175,242. D10 traite les fins ; D14 choisit entre production immédiate et fin programmée.

Consommer, produire et avancer dans la liste

_consume_and_schedule → retour dans _verify_entry → verify_trace.



Oui → Ajouter les résultats maintenant ; terminer le départ → D15.

Non → Programmer la fin à entry.cycle + process.delay → D15.

_consume_and_schedule · src/krpsim_verif/verifier.py:165-175
process.delay == 0

Pourquoi : Les besoins ont déjà été retirés. La durée décide quand les résultats sont ajoutés.
Exemple : livrer dure 0 : son résultat est disponible au prochain départ, même au même cycle.



Oui → Appeler _verify_entry pour le suivant → D09.

Non → Finaliser : _build_verified_simulator → D10, seuil 2**63 - 1.

verify_trace · src/krpsim_verif/verifier.py:243-246
for line_number, entry in enumerate(trace, start=1)

Pourquoi : On respecte l'ordre du fichier, même lorsque plusieurs départs partagent un cycle.
Exemple : E05 lance deux opérations au cycle 0 : la seconde voit le stock laissé par la première.

Construire et afficher le résultat

_build_verified_simulator / Simulator.__init__ / _print_final_stocks

Après un départ accepté

state.previous_cycle = entry.cycle
Mémoriser son moment pour vérifier le départ suivant.

state.last_cycle = max(state.last_cycle, entry.cycle)
Garder le plus grand moment pris en compte.

Après le dernier départ

_complete_until(2**63 - 1, ...)
Ajouter les résultats des fins atteignant au plus ce seuil. Voir D10.

Mettre **last_cycle** à jour avec la dernière fin ajoutée. Créer **Simulator(config)**, puis remplacer **sim.stocks**, **sim.time** et **sim._max_time** par l'état vérifié.

Ce que le constructeur fait

Il prépare des champs et crée **ProductionPlanner(config)**. Les décisions de cette préparation figurent en annexe D52-D60.

Il n'appelle pas Simulator.run. Aucune recherche de meilleure trace n'est lancée ici.

Ce qui est affiché

Après succès : **trace is valid**, puis **Final Stocks:**. Les noms sont triés alphabétiquement, alignés sur le plus long ; un stock absent vaut 0. **Last cycle:** affiche **sim.time**.

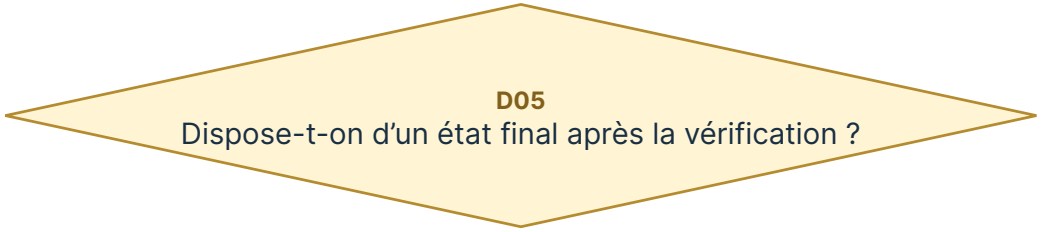
Les noms réunissent stocks initiaux, besoins et résultats : **Config.all_stock_names()**.

> **Seuil fini, pas infini.** Une opération finissant après 2**63 - 1 peut rester non créditée à la finalisation. Le cycle affiché est le maximum des départs et des fins effectivement ajoutées.

Sources : verifier.py:200-214 ; simulator.py:61-104 ; cli.py:63-71 ; parser.py:75-100.

Afficher seulement après une vérification réussie

main contrôle la présence de sim ; la conversion numérique a une limite rare.



Oui → Afficher les stocks et le dernier cycle ; retourner exit_code.

Non → Ne pas afficher de stocks ; retourner exit_code.

main · src/krpsim_verif/cli.py:98-101
sim is not None

Pourquoi : Un échec attendu renvoie None, pas un état final utilisable.
Exemple : Avec E01, afficher les stocks et le cycle 3 ; avec E02, seulement l’erreur.



Oui → Exception non interceptée par la CLI ; arrêt technique.

Non → Créer TraceEntry et continuer la lecture.

_parse_trace_line · src/krpsim_verif/verifier.py:109
int(cycle_str) peut lever ValueError

Pourquoi : isdigit accepte certains caractères que int ne sait pas convertir ; c’est une limite réelle.
Exemple : ²:scier passe isdigit mais int('²') échoue ; ٢:scier se convertit en 2.

L’atelier utilisé dans les exemples

Une configuration commune ; les variantes indiquent explicitement ce qui change.

Fichier atelier.cfg

```
bois:2
clou:8
scier:(bois:1):(planche:3):2
assembler:(planche:3;clou:4):(chaise:1):1
livrer:(chaise:1):(livraison:1):0
```

scier consomme 1 bois immédiatement et donne 3 planches après 2 cycles.

assembler consomme 3 planches et 4 clous ; la chaise arrive 1 cycle plus tard.

livrer transforme 1 chaise en 1 livraison immédiatement.

Une trace à suivre

```
0:scier
2:assembler
3:livrer
```

Les planches existent à temps pour assembler ; la chaise existe à temps pour livrer.

Commande depuis la racine du dépôt :

```
uv run krpsim_verif \
  output/pdf/eli5_source/examples/E04/atelier.cfg \
  output/pdf/eli5_source/examples/E04/trace.txt
```

Les fichiers de tous les exemples E01-E19 et E23-E28 sont livrés dans examples/. Les cas de commande E20-E22 sont reproduits par verify_examples.py.

Suivre les stocks, départ par départ

Exemple E04 exécuté : scier → assembler → livrer. Les quantités absentes valent 0.

À chaque ligne : D09 → D10 → D11 → D12 → D13 → D14 → D15. Résultat final : code 0 et **Last cycle: 3**. Une arrivée exactement au moment du départ est déjà utilisable.

Départ lu	Résultats ajoutés avant le départ	Stock disponible	Ressources nécessaires	Décision	Stock après le départ
0:scier	Aucun	bois 2 clou 8	bois 1	Accepté ; fin programmée à 2	bois 1 clou 8
2:assembler	+3 planches (scier finit à 2)	bois 1 clou 8 planche 3	planche 3 clou 4	Accepté ; fin programmée à 3	bois 1 clou 4 planche 0
3:livrer	+1 chaise (assembler finit à 3)	bois 1 clou 4 planche 0 chaise 1	chaise 1	Accepté ; résultat immédiat	bois 1 clou 4 planche 0 chaise 0 livraison 1

Comparer les traces : quand faut-il refuser ?

Configuration commune de la page « L'atelier ». Tous les cas ci-dessous ont été exécutés.

Cas / trace exacte	Configuration	Résultat observé	Ce que cela montre
E01 0:scier 2:assembler	Commune	Code 0 ; cycle 3. bois 1, clou 4, planche 0, chaise 1.	Trace partielle acceptée : aucune livraison exigée.
E02 0:assembler	Commune	Code 1 ; manque planche: 3.	D13 : les planches ne sont pas encore produites.
E03 2:scier 1:scier	Commune	Code 1 ; cycle 1 avant cycle 2.	D09 : les moments reculent.
E05 0:scier 0:scier	Commune	Code 0 ; cycle 2. bois 0, clou 8, planche 6.	Deux lancements au même cycle sont permis.
E13 0:assembler	Ajouter planche:2	Code 1 ; manque planche: 1.	D12 : manque = quantité nécessaire moins quantité disponible.
E27 0:scier 0:scier 0:scier	Commune	Code 1 au départ 3 ; manque bois: 1.	Les deux premiers départs ont consommé tout le bois.

Un fichier vide n’est pas une ligne vide

Le test de trace vide intervient après le filtrage des commentaires.

Cas / contenu de trace	Configuration	Résultat observé	Chemin à suivre
E06 : zéro octet	Commune	Code 1 ; empty trace.	D06 Oui → D07 Oui → D08 Oui.
E07 : zéro octet	Remplacer bois:2 par bois:0	Code 0 ; cycle 0 ; stocks initiaux.	D08 Non pour toutes les opérations.
E08 : zéro octet	Ajouter optimize:(chaise)	Code 0 ; cycle 0 ; stocks initiaux.	D07 Non : acceptation directe.
E09 : # commentaire suivi d’un saut de ligne	Ajouter optimize:(chaise)	Code 0 ; cycle 0.	D48 ignore le commentaire ; même liste vide que E08.
E14 : un saut de ligne seul	Commune	Code 1 ; empty trace line 1.	D47 Oui : erreur pendant la lecture.
E15 : deux espaces puis # commentaire	Commune	Code 1 ; invalid trace line 1.	D48 Non → D49 Oui : aucun deux-points.

Une production immédiate peut servir au départ suivant

E26 : deux opérations au cycle 0. Le fichier conserve l'ordre des départs.

Configuration E26

```
bois:1
scier:(bois:1):(planche:3):0
assembler:(planche:3):(chaise:1):0
```

Trace E26

```
0:scier
0:assembler
```

Résultat exécuté : code 0 ; bois 0, planche 0, chaise 1 ; dernier cycle 0.

Pourquoi cela fonctionne

1. scier consomme le bois.
2. Sa durée est nulle : D14 ajoute immédiatement 3 planches.
3. Le départ suivant reste au cycle 0 : D09 l'autorise.
4. assembler trouve les planches et les consomme ; la chaise apparaît aussitôt.

> Avec la configuration commune, la chaise n'arrive qu'au cycle 3. E28 tente livrer au cycle 2 après assembler : refus, manque chaise: 1.

E28 exécuté : trace 0:scier, puis 2:assembler, puis 2:livrer. Aucun tri ne réorganise ces départs.

Retrouver la première erreur et le bon numéro

Une erreur de syntaxe et une erreur de rejeu n'utilisent pas le même comptage.

Cas	Résultat exécuté	Explication
E18 # commentaire 0:assembler	line 1 at cycle 0 … lacks resources	line_number compte les départs : le commentaire ne compte pas. La ligne physique est pourtant la deuxième.
E19 0:assembler malforme	invalid trace line 2	idx compte les lignes du fichier. La lecture échoue avant de vérifier les ressources du premier départ.
E10 0:inconnue	unknown process 'inconnue' in trace	Le format passe ; D11 rejette ensuite le nom.
E20 : seulement le chemin de configuration	SystemExit, code 2 ; usage et erreur	La commande n'a pas reçu son fichier de trace.
E21 : chemin de trace inexistant	Code 1 ; invalid trace: …	OSError est interceptée en D04.
E22 : trace contenant les octets FF 0A	UnicodeDecodeError non interceptée	La CLI ne convertit pas cette exception en diagnostic invalid trace.

Les limites réelles du programme

Ces particularités sont décrites, sans modifier le code.

Point	Comportement vérifié ou lu dans le code
CRLF : configuration / trace	La configuration interdit tout octet CR. La trace est ouverte en mode texte : Python normalise CRLF et CR en LF. E11 (0:scier avec CRLF) est accepté.
Espaces et noms	La configuration retire les espaces aux bords de la ligne ; la trace ne les retire pas. Le nom après le premier : est conservé, autres : compris. Un nom non vide n'est pas forcément connu.
Chiffres Unicode	E23 : ²:scier passe isdigit mais int échoue (ValueError). E24 : ٢:scier devient le cycle 2 et finit à 4. Les conversions int de la configuration ont aussi des limites.
Finalisation bornée	E17 : scier dure 9223372036854775808 cycles. Trace 0:scier acceptée, bois 0, aucune planche créditée, dernier cycle 0. La fin dépasse le seuil 2**63 - 1.
État retourné	sim.stocks, sim.time et sim._max_time sont remplacés. sim.trace et sim._running restent ceux du constructeur ; ce n'est pas un historique complet prêt à reprendre la simulation.
Exceptions inattendues	ValueError, UnicodeDecodeError de trace et autres erreurs hors des except ne sont pas transformées par _verify_inputs. Un échec disque pendant la lecture de config peut aussi être étiqueté invalid trace via OSError.

Ce que « trace is valid » garantit

Valider des départs ne signifie pas certifier la meilleure solution.

Pour une trace lue depuis les fichiers

La configuration a passé ses contrôles ; les lignes de trace ont été acceptées.

Les moments de départ ne reculent pas ; les opérations citées existent.

Chaque départ disposait des ressources nécessaires après ajout des productions terminées.

Les stocks résultent du rejeu et de la finalisation bornée du code.

> Une trace partielle peut donc être valide. E12 contient seulement 0:scier : code 0, dernier cycle 2, trois planches produites.

Ce que le vérificateur ne contrôle pas

Il ne compare pas la trace à celle qu'aurait choisie krpsim.

Il ne démontre pas l'optimalité et n'exige pas l'atteinte de l'objectif.

Il ne demande pas que toutes les opérations possibles aient été exécutées.

Il ne reçoit pas le délai maximal demandé à krpsim.

Appels Python directs

verify_trace(config, trace) ne relit pas les fichiers : il suppose les objets cohérents. Il ne refait pas parse_file ou parse_trace.

API01 : un premier cycle -1 est refusé contre la référence initiale 0. Une Config construite manuellement peut aussi contourner les validations du fichier.

Configuration : choisir le fichier

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

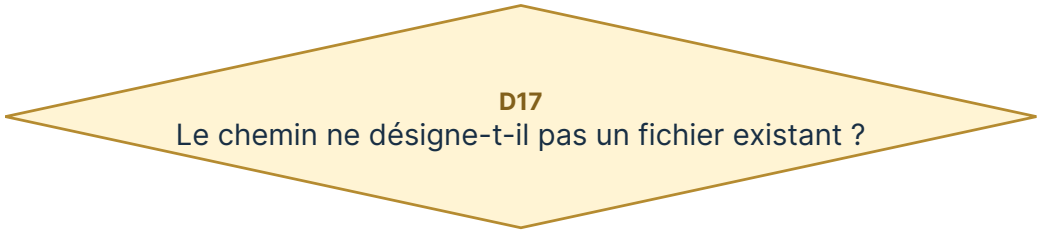


Oui → Refuser : path traversal detected → D03.

Non → Vérifier le fichier → D17.

parse_file · src/krpsim/parser.py:592-594
".." in path.parts

Pourquoi : Le programme rejette explicitement les remontées de dossier dans ce chemin.
Exemple : ../atelier.cfg est refusé avant de lire son contenu.



Oui → Refuser : invalid path → D03.

Non → Vérifier les droits → D18.

parse_file · src/krpsim/parser.py:596-598
not path.is_file()

Pourquoi : Un répertoire ou un fichier absent ne fournit pas de configuration.
Exemple : Un chemin vers un dossier est refusé.

Configuration : droits et marque invisible

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

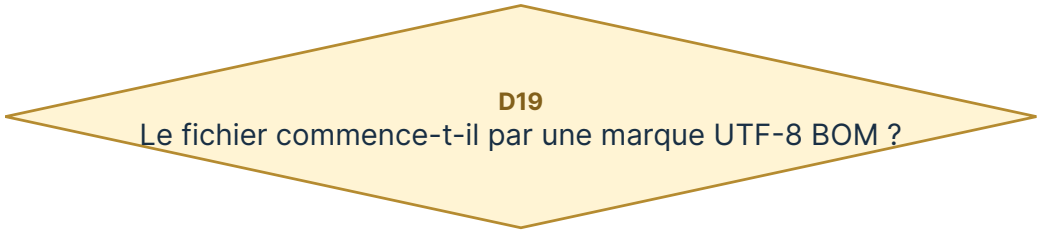


Oui → Refuser : file is not readable → D03.

Non → Lire les octets : _read_lines → D19.

parse_file · src/krpsim/parser.py:600-605
not os.access(path, os.R_OK)

Pourquoi : Les droits sont vérifiés avant l'ouverture ; un échec ultérieur reste possible.
Exemple : Sans droit de lecture, le contrôle échoue ; le test du dépôt simule ce refus.



Oui → Refuser : BOM detected in file → D03.

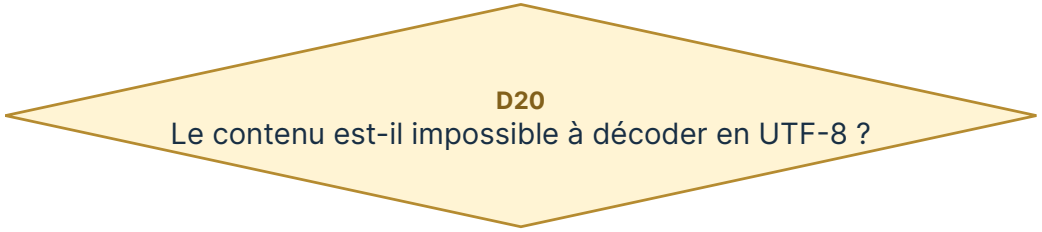
Non → Décoder le texte → D20.

_read_lines · src/krpsim/parser.py:487-491
raw.startswith(UTF8_BOM)

Pourquoi : Cette marque invisible est interdite dans la configuration.
Exemple : Les trois premiers octets EF BB BF déclenchent le refus.

Configuration : encodage et fins de ligne

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

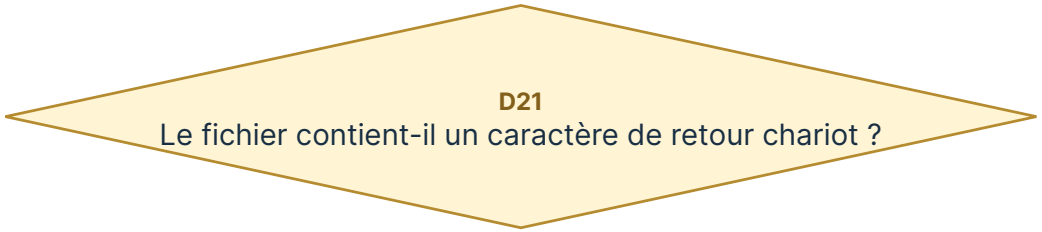


Oui → Convertir en ParseError : file must be UTF-8 encoded → D03.

Non → Examiner les fins de ligne → D21.

_read_lines · src/krpsim/parser.py:493-499
except UnicodeDecodeError as exc

Pourquoi : Le programme doit pouvoir lire les caractères de la configuration.
Exemple : L'octet FF isolé n'est pas un texte UTF-8 valide.



Oui → Refuser : CRLF line endings are not allowed → D03.

Non → Séparer les lignes et vérifier leur longueur → D22.

_read_lines · src/krpsim/parser.py:501-505
b"\r" in raw

Pourquoi : La configuration interdit tout retour chariot, même isolé.
Exemple : Une configuration avec des fins Windows CRLF est refusée ; une trace CRLF est acceptée.

Configuration : longueur et commentaires

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.



Oui → Refuser : line exceeds 255 characters → D03.

Non → Ligne suivante ; après toutes, _parse_lines → D23.

_read_lines · src/krpsim/parser.py:507-513
len(raw_line) > 255

Pourquoi : La longueur est contrôlée avant de supprimer espaces ou commentaires.
Exemple : Une ligne de commentaire de 256 caractères est refusée ; 255 passe ce contrôle.



Oui → Ignorer et lire la ligne suivante ; fin du fichier → D43.

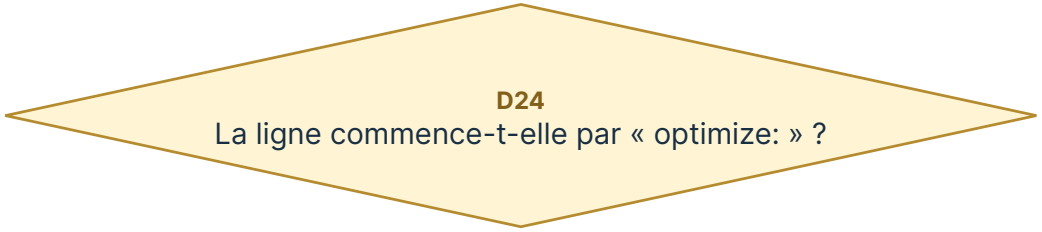
Non → Identifier son type → D24.

_parse_lines · src/krpsim/parser.py:546-552
not line or line.startswith("#")

Pourquoi : Les lignes de décoration de la configuration ne décrivent pas de données.
Exemple : Une ligne « # atelier » est ignorée ici ; les règles de trace sont différentes.

Configuration : reconnaître le type de ligne

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.



Oui → Appeler _handle_optimize → D38, puis ligne suivante.

Non → Chercher une déclaration d'opération → D25.

_parse_lines · src/krpsim/parser.py:554-556
line.startswith("optimize:")

Pourquoi : Ce préfixe est reconnu avant les autres formes de lignes.
Exemple : optimize:(chaise) indique un objectif ; sa validité sera contrôlée.



Oui → Appeler _handle_process → D31, puis ligne suivante.

Non → Chercher une déclaration de stock → D26.

_parse_lines · src/krpsim/parser.py:558-560
":(" in line

Pourquoi : Le tri initial choisit le lecteur spécialisé ; il ne valide pas encore toute la ligne.
Exemple : scier:(bois:1):(planche:3):2 est envoyé au lecteur des opérations.

Configuration : lire un stock initial

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

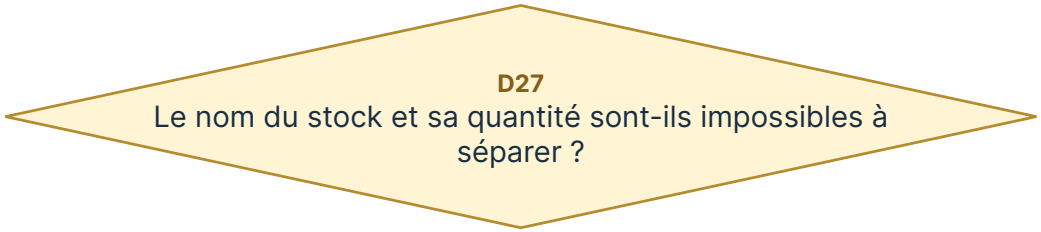


Oui → Appeler _handle_stock → D27, puis ligne suivante.

Non → Refuser : unrecognized line → D03.

_parse_lines · src/krpsim/parser.py:562-568
":" in line

Pourquoi : Sans forme reconnue, la configuration est refusée.
Exemple : bois:2 décrit un stock ; bonjour seul est refusé.



Oui → Refuser : invalid stock line → D03.

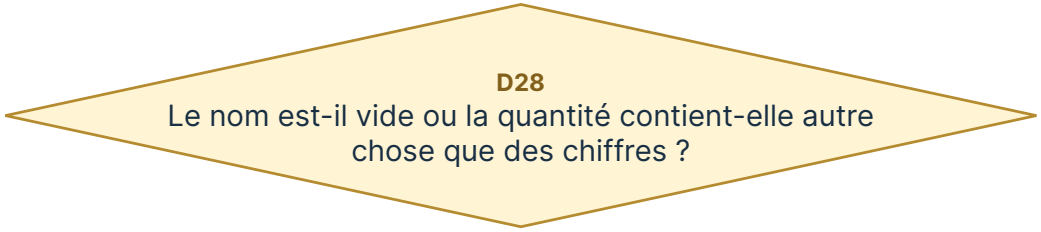
Non → Vérifier les deux morceaux → D28.

_parse_stock · src/krpsim/parser.py:127-133
except ValueError : line.split(":", 1)

Pourquoi : Cette protection couvre aussi les appels directs à _parse_stock.
Exemple : _parse_stock('bois') échoue ; la commande l'aurait déjà rejeté en D26.

Configuration : lire la quantité initiale

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

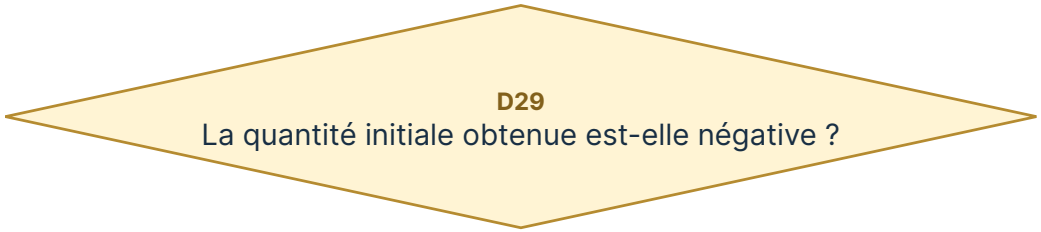


Oui → Refuser : invalid stock line → D03.

Non → Convertir qty avec int ; contrôler D29.

_parse_stock · src/krpsim/parser.py:135-139
not name or not qty.isdigit()

Pourquoi : Un stock doit avoir un nom et une quantité lisible comme entier.
Exemple : bois:x et :2 sont refusés. Certains chiffres Unicode passent isdigit mais pas int.



Oui → Refuser : invalid stock quantity → D03.

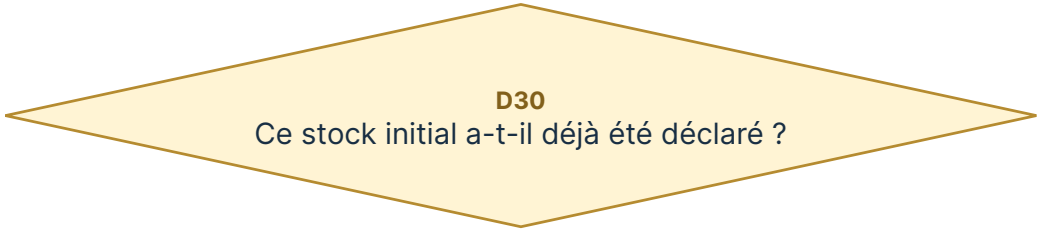
Non → Renvoyer (name, quantity) ; contrôler D30.

_parse_stock · src/krpsim/parser.py:141-145
quantity < 0

Pourquoi : Contrôle défensif présent dans le code, normalement inaccessible après isdigit et int.
Exemple : bois:-1 est déjà refusé en D28. Le test du dépôt force ce chemin avec un substitut de chaîne.

Configuration : stock unique et recette valide

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

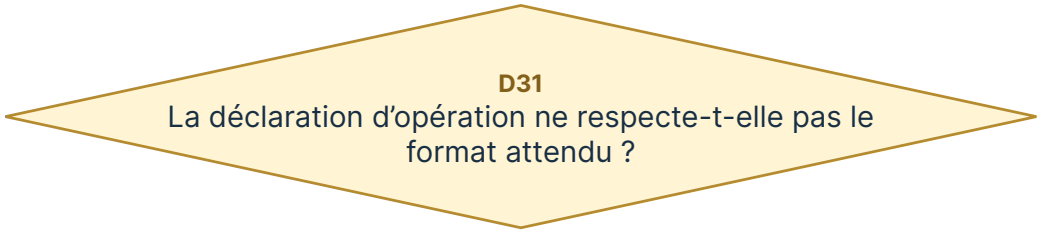


Oui → Refuser : duplicate stock → D03.

Non → Mémoriser stocks[name] = qty ; ligne suivante → D23.

_handle_stock · src/krpsim/parser.py:370-376
name in stocks

Pourquoi : Une ressource ne doit pas avoir deux quantités initiales concurrentes.
Exemple : Deux lignes bois:2 puis bois:3 sont refusées, pas additionnées.



Oui → Refuser : invalid process line → D03.

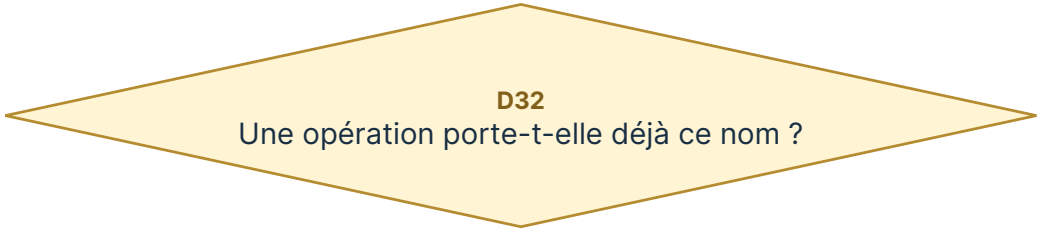
Non → Lire durée ; _parse_resources pour besoins puis résultats → D33.

_parse_process · src/krpsim/parser.py:219-243
not match

Pourquoi : La forme attendue est nom:(besoins):(résultats):durée ; la durée peut valoir 0.
Exemple : scier:(bois:1):(planche:3):2 passe ; une durée -2 échoue.

Configuration : opération unique et bloc vide

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

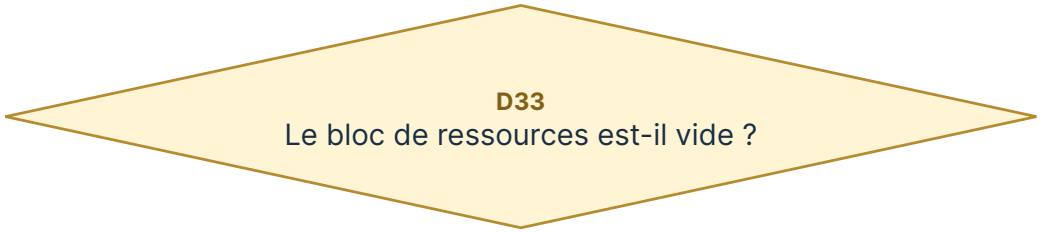


Oui → Refuser : duplicate process → D03.

Non → Ajouter processes[process.name] ; ligne suivante → D23.

_handle_process · src/krpsim/parser.py:342-348
process.name in processes

Pourquoi : Ce contrôle arrive après la lecture complète de l'opération.
Exemple : Déclarer scier deux fois est refusé, même si les recettes diffèrent.



Oui → Renvoyer {} au lecteur ; après besoins et résultats, contrôler D32.

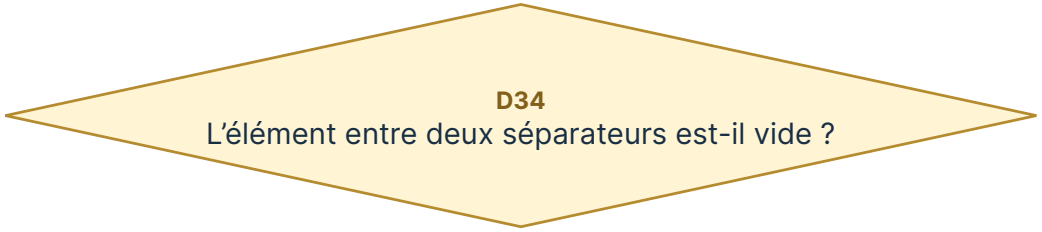
Non → Parcourir les éléments séparés par ; → D34.

_parse_resources · src/krpsim/parser.py:167-173
not block

Pourquoi : Une opération peut ne rien consommer ou ne rien produire.
Exemple : cadeau:():(planche:1):0 n'a aucun besoin ; un résultat omis devient aussi un bloc vide.

Configuration : lire chaque ressource

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

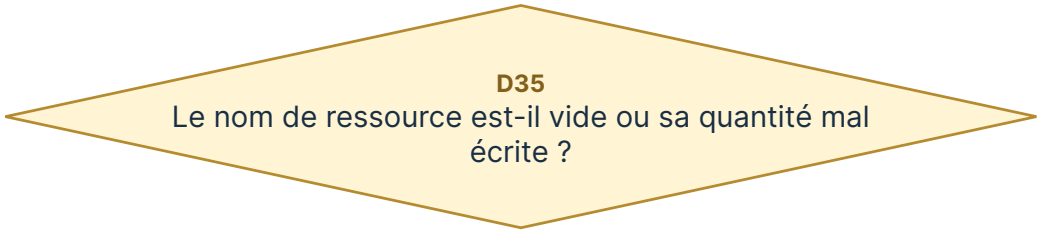


Oui → Ignorer cet élément ; passer au suivant.

Non → Séparer nom et quantité → D35.

_parse_resources · src/krpsim/parser.py:173-179
not item

Pourquoi : Les séparateurs supplémentaires sont tolérés dans les blocs.
Exemple : bois:1;;clou:4 produit les mêmes ressources que bois:1;clou:4.



Oui → Refuser : invalid quantity for resource → D03.

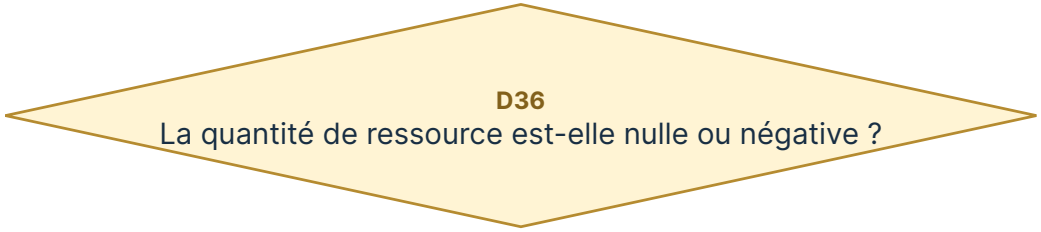
Non → Convertir avec int ; contrôler D36.

_parse_resources · src/krpsim/parser.py:179-185
not name or not qty.isdigit()

Pourquoi : Chaque élément doit nommer une ressource et préciser sa quantité.
Exemple : bois:x, :2 et bois sans quantité sont refusés.

Configuration : quantités et doublons

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

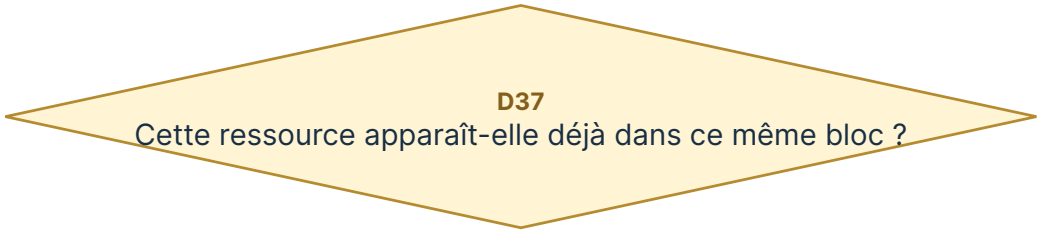


Oui → Refuser : invalid quantity for resource → D03.

Non → Vérifier le doublon → D37.

_parse_resources · src/krpsim/parser.py:187-189
quantity <= 0

Pourquoi : Les quantités d'une recette doivent être strictement positives, contrairement au stock initial.
Exemple : bois:0 est permis comme stock initial, mais refusé dans les besoins d'une opération.



Oui → Refuser : duplicate resource → D03.

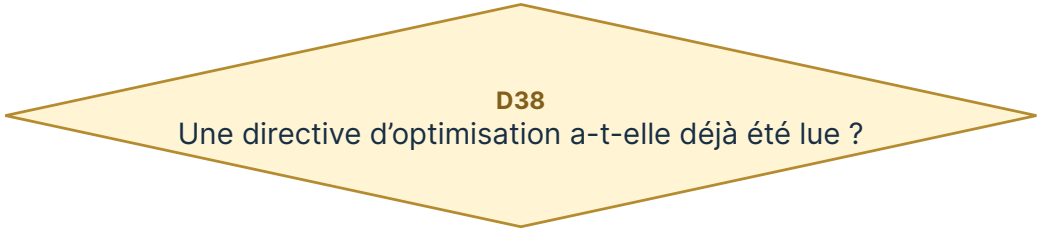
Non → Ajouter ; élément suivant → D34. Bloc terminé : retourner au lecteur.

_parse_resources · src/krpsim/parser.py:191-197
name in resources

Pourquoi : Un besoin ou un résultat ne peut pas nommer deux fois la même ressource dans son bloc.
Exemple : (bois:1;bois:2) est refusé. Du bois dans les besoins ET les résultats est permis.

Configuration : une directive bien formée

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.



Oui → Refuser : duplicate optimize line → D03.

Non → Appeler _parse_optimize → D39.

_handle_optimize · src/krpsim/parser.py:316-320
optimize is not None

Pourquoi : Le fichier ne peut déclarer qu'une seule liste d'objectifs.
Exemple : Deux lignes optimize:(chaise) sont refusées.



Oui → Refuser : invalid optimize line → D03.

Non → Extraire le bloc ; filtrer les éléments → D40.

_parse_optimize · src/krpsim/parser.py:266-274
not match

Pourquoi : Les objectifs doivent être placés entre parenthèses.
Exemple : optimize:chaise échoue ; optimize:(chaise) passe.

Configuration : conserver les objectifs utiles

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.



Oui → Conserver ; élément suivant. Après tous : D41.

Non → Ignorer l'élément ; après filtrage → D41.

_parse_optimize · src/krpsim/parser.py:274
[item for item in block.split(";") if item]

Pourquoi : Les éléments vides entre séparateurs ne deviennent pas des objectifs.
Exemple : optimize:(;chaise;;) conserve uniquement chaise.



Oui → Refuser : invalid optimize line → D03.

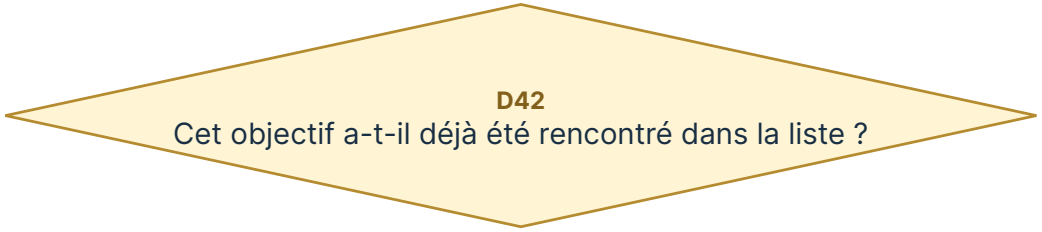
Non → Parcourir les objectifs et chercher les doublons → D42.

_parse_optimize · src/krpsim/parser.py:276-284
not items

Pourquoi : Une directive présente doit demander au moins un objectif.
Exemple : optimize:() et optimize:(;;) sont refusés.

Configuration : objectifs uniques et sections présentes

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

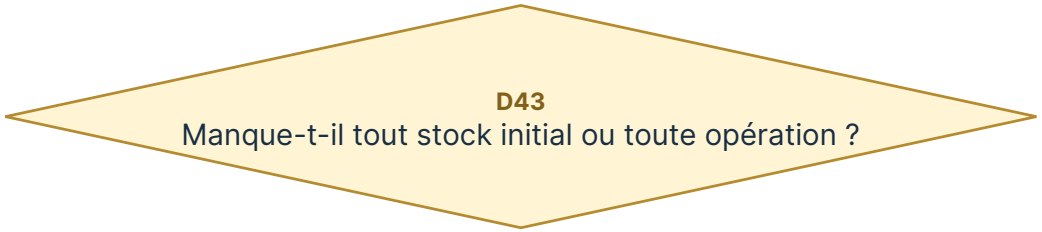


Oui → Refuser : duplicate optimize target → D03.

Non → Ajouter à seen et result ; objectif suivant. Après tous : ligne suivante → D23.

_parse_optimize · src/krpsim/parser.py:284-294
item in seen

Pourquoi : L'ordre est conservé, sans doublon.
Exemple : optimize:(chaise;chaise) est refusé ; optimize:(time;chaise) conserve cet ordre.



Oui → Refuser : configuration must define at least one stock and process → D03.

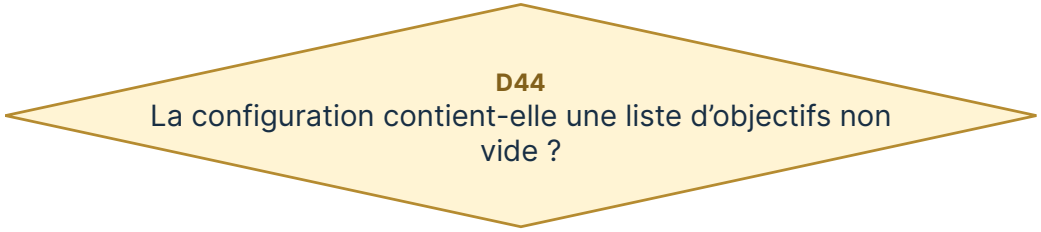
Non → Contrôler les objectifs → D44.

parse_file · src/krpsim/parser.py:607-611
not stocks or not processes

Pourquoi : Une configuration lue depuis un fichier doit contenir ces deux catégories.
Exemple : bois:2 seul est insuffisant ; bois:0 compte bien comme un stock déclaré.

Configuration : contrôler les objectifs

Sous-schémas de parse_file et de ses fonctions déléguées · Tout ParseError revient à D03.

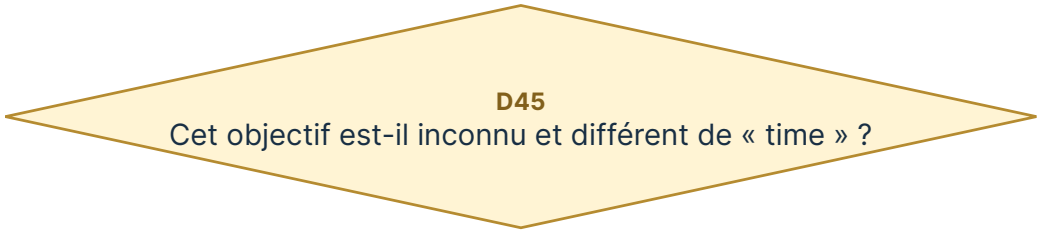


Oui → Appeler _validate_optimize → D45, puis D46.

Non → Vérifier les ressources des opérations → D46.

parse_file · src/krpsim/parser.py:613-617
if optimize

Pourquoi : La validation des cibles précède celle des ressources nécessaires.
Exemple : Sans ligne optimize, ce contrôle est sauté.



Oui → Refuser : unknown stock … in optimize line → D03.

Non → Objectif suivant ; après tous, passer à D46.

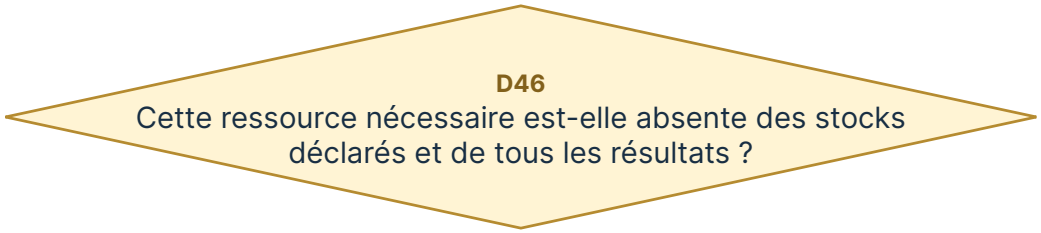
_validate_optimize · src/krpsim/parser.py:406-418
item != "time" and item not in known

Pourquoi : known réunit les stocks, besoins et résultats ; time est un nom spécial accepté.
Exemple : optimize:(licorne) échoue si aucune ressource ne s'appelle licorne.

Configuration : une source pour chaque besoin

_validate_process_resources · Après ce contrôle, retourner Config et commencer parse_trace.

> Après la dernière ligne : D43 → D44 → éventuellement D45 → D46. L'existence d'un producteur ne prouve pas qu'il pourra démarrer : la vérification des stocks vient plus tard, à chaque départ.



_validate_process_resources · src/krpsim/parser.py:449-463
res not in defined

Pourquoi : Ce contrôle vérifie l'existence d'une source, pas la disponibilité actuelle ni l'absence de cycle.
Exemple : planche peut manquer au départ si scier sait en produire. Le départ devra encore attendre.

Les boucles : où revient-on ?

Un renvoi « suivant » reprend le même contrôle sur un nouvel élément.

Parcours	Action répétée	Sortie de la boucle
_read_lines / _parse_lines	Longueur de chaque ligne ; classification et lecture des lignes utiles.	Toutes lues → contrôle global D43.
_parse_resources / _parse_optimize	Éléments séparés par ; puis cibles conservées.	Retourner le dictionnaire / la liste au lecteur appelant.
_validate_optimize / _validate_process_resources	Chaque cible ; chaque besoin de chaque opération.	Retourner Config après tous les contrôles.
parse_trace	Chaque ligne physique : ignorer ou créer TraceEntry.	Retourner entries → D06.
_verify_empty_trace	Chaque opération et chacun de ses besoins (all).	Premier démarrage possible : erreur ; sinon acceptation.
verify_trace	Chaque départ, dans l'ordre ; arrêt à la première exception.	Finalisation après le dernier départ.
_complete_until	Retirer la fin la plus proche, ajouter tous ses résultats.	Aucune fin restante ou prochaine fin après le seuil.
_missing_resources / _consume_and_schedule	Chaque besoin ; puis les résultats si durée nulle.	Dictionnaire missing / retour vers _verify_entry.
Constructeurs / affichage	Cibles, opérations, résultats ; noms de ressources triés.	Objet prêt / dernière ligne affichée.

Annexe : préparer l'objet retourné

Simulator.__init__ → ProductionPlanner.__init__ · Préparation uniquement, sans recherche de plan.

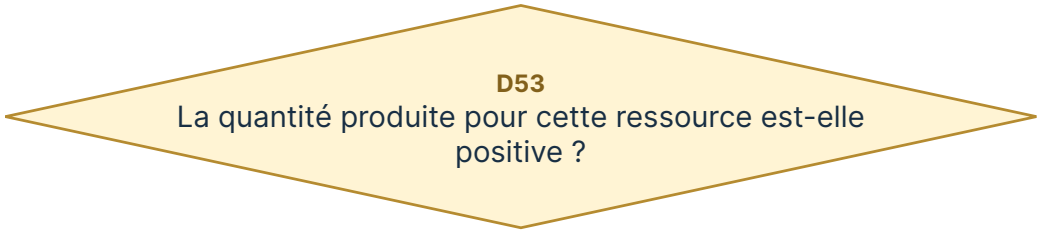


Oui → Conserver le nom dans self.targets.

Non → L'exclure ; continuer la liste.

ProductionPlanner.__init__ · src/krpsim/optimizer.py:62
t != "time"

Pourquoi : Le constructeur prépare ses données même si aucune recherche de plan n'est lancée.
Exemple : [time, chaise] devient (chaise,) dans self.targets.



Oui → Ajouter cette opération à self._producers[resource].

Non → Ne pas l'ajouter ; résultat suivant.

ProductionPlanner.__init__ · src/krpsim/optimizer.py:64-68
quantity > 0

Pourquoi : Ce répertoire indique quelles opérations produisent chaque ressource.
Exemple : scier produit 3 planches : scier est enregistré comme producteur de planche.

Annexe : reconnaître une conversion finale

ProductionPlanner._terminal_converter · L'échec de reconnaissance renvoie None, pas une erreur de trace.

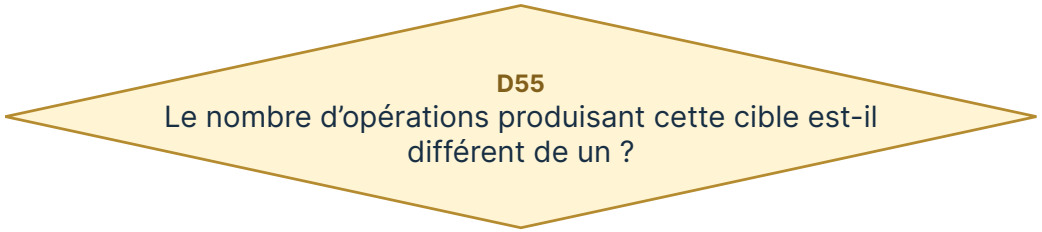


Oui → Retourner None ; choisir goal → D59.

Non → Chercher les producteurs de cet objectif → D55.

_terminal_converter · src/krpsim/optimizer.py:84-87
len(self.targets) != 1

Pourquoi : La détection du convertisseur final ne s'applique qu'à une seule cible.
Exemple : Aucun objectif, ou deux objectifs ressources : aucun convertisseur final.



Oui → Retourner None → D59.

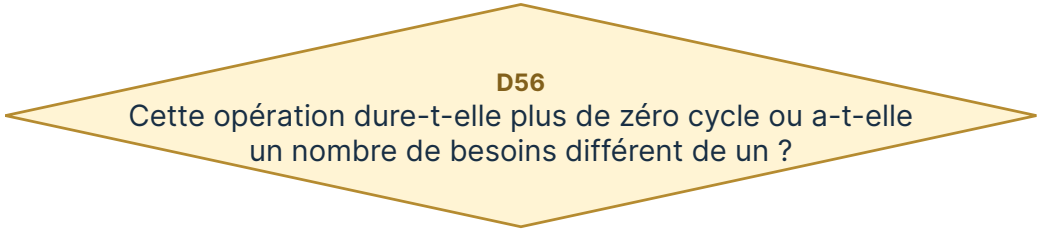
Non → Examiner l'unique producteur → D56.

_terminal_converter · src/krpsim/optimizer.py:88-90
len(candidates) != 1

Pourquoi : Le constructeur n'identifie un convertisseur final que s'il est unique.
Exemple : Deux opérations produisant chaise empêchent cette reconnaissance.

Annexe : vérifier la forme du convertisseur

ProductionPlanner._terminal_converter · Ces décisions concernent les champs du constructeur.

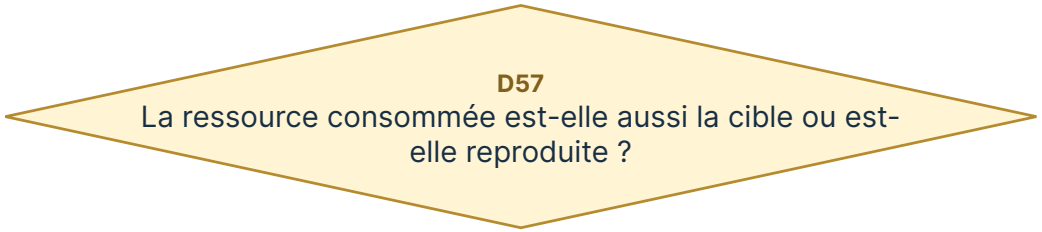


Oui → Retourner None → D59.

Non → Examiner sa seule ressource consommée → D57.

_terminal_converter · src/krpsim/optimizer.py:91-93
process.delay != 0 or len(process.needs) != 1

Pourquoi : Pour les configurations valides, le convertisseur recherché est immédiat et consomme une seule ressource.
Exemple : assembler dure 1 cycle et consomme deux ressources : ce n'est pas ce convertisseur.



Oui → Retourner None → D59.

Non → Chercher un producteur de la ressource consommée → D58.

_terminal_converter · src/krpsim/optimizer.py:94-95
source == target or process.results.get(source, 0) > 0

Pourquoi : Cette reconnaissance écarte les opérations qui réutilisent la même ressource de cette façon.
Exemple : Une conversion qui consomme chaise et en rend aussi est écartée.

Annexe : choisir le but interne

ProductionPlanner._terminal_converter → ProductionPlanner.__init__.

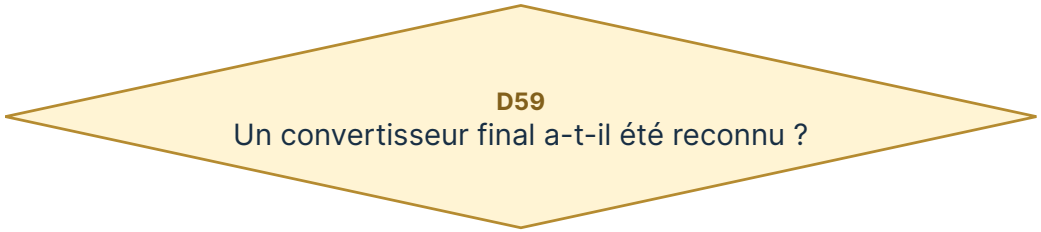


Oui → Retourner None → D59.

Non → Retourner process comme convertisseur final → D59.

_terminal_converter · src/krpsim/optimizer.py:96-98
not self._producers.get(source)

Pourquoi : Le constructeur recherche une étape finale alimentée par une autre opération.
Exemple : livrer consomme chaise, que assembler produit : ce contrôle passe.



Oui → goal = nom de son unique besoin ; finir le constructeur.

Non → Choisir un objectif de repli → D60.

ProductionPlanner.__init__ · src/krpsim/optimizer.py:68-72
self.terminal is not None

Pourquoi : Ces champs sont initialisés, sans ordonner ni lancer des opérations pour la vérification.
Exemple : Si livrer est reconnu, goal vaut chaise, sa ressource consommée.

Annexe : objectif de repli

ProductionPlanner.__init__ · Ce champ n'est pas une preuve d'objectif atteint.

> Après le constructeur, la trace vide renvoie l'état initial. La trace non vide remplace les stocks et le temps de l'objet par le résultat du rejeu. Les méthodes de recherche, de simulation et d'optimisation ne sont pas appelées par ce parcours.



ProductionPlanner.__init__ · src/krpsim/optimizer.py:72
self.targets

Pourquoi : Le repli conserve la première cible, ou aucune s'il n'y en a pas.
Exemple : Sans optimize, targets est vide et goal vaut None.

Corrections apportées au document précédent

Comparaison avec krpsim_verif_arbre_de_decision.pdf, édition du 31 octobre 2026.

Emplacement ancien	Correction vérifiée
Page 2 : CRLF laisse un caractère CR	Faux pour parse_trace : l'ouverture texte normalise les fins de ligne. Le test test_parse_trace_accepts_crlf et E11 confirment l'acceptation.
Page 2 : ouverture / décodage → OSError	Les deux ne se confondent pas : UnicodeDecodeError de trace n'est pas interceptée par la CLI. E22 le confirme.
Pages 3 et 9 : trace non vide → construire Simulator	Ce raccourci doit passer par tous les départs et leur vérification avant la construction finale. Le nouveau parcours renvoie vers D09-D15.
Pages 4 et 10 : retour de Process après les besoins	Les fonctions sont distinctes : _resolve_process renvoie Process ; _missing_resources renvoie toujours un dictionnaire, éventuellement vide.
Page 5 : départ programmé	Après le lancement, ce qui est mis dans le tas est la fin future ; les ressources ont déjà été consommées au départ.
Page 11 : référence verifier.py:158-175	La boucle de complétion se trouve actuellement aux lignes 123-136 ; 158-175 décrit _consume_and_schedule.

Retrouver les preuves et régénérer le guide

La matrice complète est livrée dans matrice_tracabilite.csv.

60 décisions repérées

D01-D05 : commande et diagnostic.

D06-D15 : trace vide, chronologie, ressources et production.

D16-D46 : lecture et validation de configuration.

D47-D51 : lecture de trace et conversion numérique.

D52-D60 : initialisation du planificateur dans l'objet retourné.

Les boucles et les actions sans condition sont décrites dans les pages de parcours. D01-D04, D15, D20, D27 et D51 représentent aussi des choix de bibliothèque, parcours ou exceptions : ce ne sont pas tous des « if » littéraux.

Fichiers éditables

content.json : textes, décisions, renvois et preuves.

style.css : mise en page et charte graphique.

build.py / guide.html : génération et version HTML consultable.

verify_examples.py / example_results.json : exécution et observations.

examples/ : configurations et traces exactes.

README.md / validation.md : régénération, versions et bilan des contrôles.

> La matrice associe chaque question à sa condition, sa fonction, ses lignes, ses sorties et un test ou exemple. Une preuve « lecture du code » n'est pas présentée comme un test exécuté. La couverture documentaire ne remplace pas la couverture des tests.

Ce qui a été vérifié

Bilan de génération · Les limites du dépôt sont distinctes de la validation documentaire.

Document et exemples

60 décisions inventoriées et associées à leurs sources ; matrice CSV livrée.

30 cas exécutés, dont 28 cas E01-E28, un exercice EX02 et un appel direct API01. Les codes, erreurs attendues, stocks finaux et cycles sont contrôlés selon le cas.

Les pages sont rendues en images, relues visuellement et contrôlées pour les débordements. Les liens internes et l'intégration d'Inter sont vérifiés.

Les empreintes des sources et les versions des outils figurent dans **validation.md** et les fichiers de preuves.

Contrôles du dépôt sous Python 3.10

Analyse de types, lint, sécurité, complexité et construction du paquet : contrôles exécutés ; détail dans le bilan joint.

La CI du dépôt n'est pas entièrement validée. La suite brute rencontre des chemins de ressources obsolètes : les fichiers sont dans good_resources et bad_resources.

Le contrôle Black signale aussi un formatage préexistant dans src/krpsim/cli.py. Aucun seuil ni fichier du programme n'a été modifié pour masquer ces échecs.

Aucune validation de compréhension auprès d'un lecteur débutant n'est revendiquée.

À vous de suivre les décisions

Utilisez la configuration commune de la page « L'atelier ». Corrections sur la page suivante.

1. Cette trace est-elle acceptée ?

```
0:scier
2:assembler
3:livrer
```

Indiquez le code de sortie et le dernier cycle. Expliquez quand les planches et la chaise deviennent disponibles.

2. Quelle erreur arrive en premier ?

```
2:scier
1:inconnue
```

Le second départ a deux problèmes : son moment recule et son nom est inconnu. Quelle vérification arrête le programme ?

3. Quels stocks après deux départs ?

```
0:scier
0:scier
```

Calculez les stocks immédiatement après le second départ, puis après la finalisation.

Combien de bois reste-t-il ? Combien de planches sont disponibles à chacun de ces moments ? Quel est le dernier cycle affiché ?

> Pour chaque réponse, citez au moins deux identifiants Dxx et retrouvez les fonctions dans le code. L'objectif est de justifier votre réponse, pas de deviner le résultat.

Corrections des trois exercices

Autoévaluation : ces exercices n'ont pas fait l'objet d'un test de compréhension avec un lecteur débutant.

1. Acceptée : code 0, dernier cycle 3

Les moments 0, 2 et 3 ne reculent pas : **D09**. Au cycle 2, **D10** ajoute les trois planches de scier ; assembler peut les consommer. Au cycle 3, la chaise est ajoutée avant livrer. Sa durée nulle fait produire la livraison immédiatement : **D14 Oui**. Exemple exécuté E04.

2. Le recul du cycle est signalé en premier

_verify_entry teste **D09** avant d'appeler **_resolve_process** et **D11**. Le message est « line 2: cycle 1 is before cycle 2 ». Le programme ne recherche donc pas le nom inconnue pour ce départ. Ce cas est fourni dans le dossier EX02.

3. Deux temps à distinguer

Juste après le second départ : **bois 0, clou 8, planche 0** (absente du dictionnaire). Les deux sciages sont encore en cours. **D14 Non** a programmé deux fins au cycle 2.

Après finalisation : **bois 0, clou 8, planche 6**. **D10** ajoute les résultats des deux opérations. Le dernier cycle affiché vaut **2**. Exemple exécuté E05.

> Vous avez compris le parcours si vous pouvez expliquer la différence entre « consommer au départ » et « produire à la fin », puis retrouver ces actions dans _consume_and_schedule et _complete_until.